

RRDS26 Workshop 2 Activity 2

Git and GitHub

Setup

1. Install GitHub Desktop.
2. If you don't have one already, create an account on GitHub. Remember that the name should be fairly professional and be able to identify you reasonably well.

Fun with Git and GitHub

3. On GitHub, create a new repository called RRDS26-Workshop2. Make it public (you can always change this later). Go ahead and select the option to add a README and a .gitignore (you can select R if you are an R user). You don't need to add a license at this time, but you can if you'd like.
4. Open GitHub Desktop, sign into your GitHub account, and select the option to Clone a Repository. Select the RRDS26-Workshop2 from the list, choose a local path where you want it to be saved on your local computer (I suggest a folder inside a parent folder for this conference like RRDS26), and click Clone.
5. Make sure your computer is showing your hidden files (there is a slide explaining how to do that). There should be a .git folder, a .gitignore file, and a README.md file in the repo you just cloned.
6. Edit the README.md file by adding a description about this being your repository for Activity 2 for this workshop at the Red Rock Data Science conference 2026. You can edit this in a text editor like TextEdit or Notepad or you can edit it in RStudio or Positron. Note that it is standard practice to end the file with a new line, so press Enter/Return at the end of the last line. GitHub Desktop will make note if the file does not end with a new line.
7. Edit the .gitignore file. Then add a line that has ".DS_Store" without the quotes. Files that end in .DS_Store are created by some operating systems (like macOS) for file management. We don't want that file tracked by Git.
8. Now, in GitHub Desktop, there should be tracked changes for both the README and .gitignore files. Let's commit those changes. In the Summary box on the bottom left, put a short summary of the changes and, if desired, a longer description. Then click Commit 2 files to main.
9. Now create a directory in this RRDS-Workshop2 folder called Activities. Add the Activity 1 Quarto and html files you created earlier into that folder.
10. Add and commit again to indicate the Activity 1 has been added. Then Push those changes to GitHub.

11. Now, near the top left of the GitHub Desktop window, click on History. This will show all your commits on this branch. Click on the Initial commit.
12. Right click on the Initial commit and select Create Branch from Commit. Name this something (like branch2). You'll see the Current Branch at the top of the page is showing your new branch name rather than main. If you navigate back to your folder on your local computer, you'll see that the Activities folder is gone and the README and .gitignore are restored to their past versions. This has put you in a new temporary branch. Click on the Current Branch button at the top of GitHub Desktop to see the two branches. You can switch back and forth between them (choose to leave the second branch's work as it is) and the contents in your folder will change depending on which branch is active.
13. Now, while on the new branch (not main) create a new folder in your local RRDS26-Workshop2 directory called Activities and copy these Activity 2 Instructions into that folder. Then add and commit that change (don't push or publish it).
14. Switch back to the main branch. We are going to merge the branches now. From the Branch menu, select Merge into Current Branch. Then select your second branch and click Create a merge commit. This should bring the Activity 2 Instructions document into your main branch.
15. **SKIPABLE (More Advanced)**: That merge was very clean because the second branch only had new files. It is less clean when a previously existing file is edited on the second branch.
 - a. Switch back to the second branch, then edit the README.md file to include a new line that was not in the README on the main branch. Save and Commit that README.
 - b. Switch back to the main branch and again select the Branch menu then Merge Into Current Branch then your second branch. It will say there is 1 conflicted file. Click on Create a merge commit. Then you will have the option to open that file to resolve the conflict. Do that and delete the lines you don't want to keep in that file. Note that HEAD is your main branch.
 - c. Once you've saved the README, you can continue the merge in GitHub Desktop. That will complete the merge conflict.
16. We can now delete the second branch by clicking on the Current Branch button, right clicking on the second branch, and selecting delete.
17. Now push all these commits from the main branch to GitHub. Visit your GitHub page. All the files should be there.

Forking

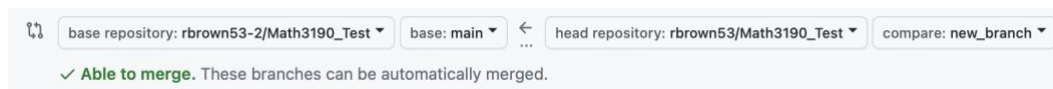
18. Let's now practice forking. Navigate to my GitHub page: <https://github.com/rbrown53>. Click on Repositories and then on the RRDS26 repo. Near the top on the right side, click on Fork. You can keep all options as they are (add a description if you'd like) and then click on Create Fork.

19. On GitHub, you will now have a RRDS26 repo that is forked from mine. You could then clone it down to your local computer if you'd like.

Pull Request **SKIPABLE (More Advanced)**

20. When you fork a repository, you can make changes to it in the main branch or a new one. Once you have, you can initiate a pull request (PR) for the original repo owner to incorporate your changes. Here is how it works:

- Clone the forked RRDS26 repo to your local computer.
- Create a new branch (it is good practice to leave the main branch identical to the one you forked).
- Make a change to the RRDS26 repo (any change is fine) and push it to GitHub. It will create a new branch on GitHub as well.
- Go to GitHub and to the forked RRDS26 repo (yours, not mine). At the top, it should indicate that your repo is a couple commits ahead of mine. Click on that message. This will take you to the Pull Request (PR) page. Near the top, you should see something that looks like the picture below. Of course, in your case, the left side will have rbrown53/RRDS26 and the right side will have your username. The arrow in the middle is indicating the repo on the right will be sent to the repo on the left.



- Add a title to your PR that indicates what you did (added a new folder). Then add a description with more detail of what is in it. Then click on Create pull request. You can check out the Files changed page for a summary of what is different in the original and in the directory you requested be pulled.

I am not going to accept this pull request, but you can see how the process works.

DO NOT DO THE FOLLOWING. I am including this for completion so you know what the next steps usually would be. Usually, if the pull request was accepted, it would be best go back to the forked repository, select the main branch, and click the Sync fork button and then Update branch. This will align your fork with my original repo (after the PR was accepted). After that, one would typically go back to GitHub Desktop, switch back to the main branch, and use click the Fetch Origin button and then pull the changes. This will update your local directory with GitHub, which is synced to my original repo. Finally, you would delete the new branch since main is up to date. It is usually best to not delete this until the PR has been accepted and then the main branch is updated.